Adaptive Data Movement Strategy Selection for AES and ECDSA Accelerator Workloads on STM32H7-Class MCU

Süleyman Ege Karakaya
Computer Engineering Department Hacettepe University, Ankara, Turkiye
Graduate School of Science and Engineering Hacettepe University, Ankara, Turkiye
segekarakaya26@hacettepe.edu.tr

***Abstract*—Cryptographic acceleration is common in secure embedded systems, but the end-to-end performance of such systems is often dominated by data movement rather than raw algorithmic execution. This proposal studies how an STM32H7 class microcontroller should choose among different transfer mechanisms when invoking AES and ECDSA accelerator workloads under changing payload sizes and memory contention. The target system consists of an STM32H745 class MCU, on-chip SRAM, DMA support, and a memory-mapped external or tightly coupled cryptographic accelerator interface. The proposed work formulates transfer-mode selection as an optimization problem over latency, CPU occupancy, and bus pressure. Candidate modes include programmed I/O, DMA single-shot, DMA burst with batching, and double buffering when supported. The project will build a structured performance model, collect workload traces, and evaluate a runtime selector based on either rule-based thresholds or a lightweight classifier trained offline. The expected contribution is a system-level methodology that identifies when each transfer strategy is preferable and quantifies the benefit of adaptive selection over static policies in constrained embedded cryptographic systems.**

***Index Terms*—embedded systems, cryptographic accelerator, DMA, AES, ECDSA, data movement optimization**

I. Introduction

Modern embedded platforms increasingly rely on hardware-assisted cryptography for secure boot, firmware authentication, secure storage, and protected communication. In these systems, standardized primitives such as AES and ECDSA are attractive because they are well understood, widely deployed, and supported by both software and hardware implementations [1], [2]. However, accelerator-centric performance discussions often focus on the compute core alone, even though end-to-end latency also depends on how input data, descriptors, control words, and results move across the processor, memory system, and accelerator interface.

This problem is especially relevant in MCU class systems. Compared with application processors, STM32H7 class devices operate under tighter memory, buffering, and bus contention constraints, yet they still need to serve diverse cryptographic workloads. Bulk encryption jobs such as AES over packet or file buffers benefit from streaming and overlap, while ECDSA signing or verification involves smaller but more control-intensive transactions. A fixed transfer policy is therefore rarely optimal across all cases.

The central question of this proposal is the following: *given a workload type, payload size, and contention level, which transfer strategy minimizes overall cost when a cryptographic accelerator is invoked from an STM32H7 class embedded system?* To answer this question, the proposal treats transfer-mode selection as an optimization problem rather than as a purely implementation convenience. The work is positioned in the Optimization Track, but it remains grounded in a concrete embedded system architecture so that the eventual results can guide practical design choices.

II. Related Work

This review organizes the literature around three themes that are directly relevant to the proposed problem: accelerator-centric cryptographic design, DMA/interconnect-aware data movement, and adaptive hardware/software execution. The reviewed set emphasizes recent peer-reviewed studies, complemented by foundational standards and vendor architecture documents where they help define the design space of MCU-class embedded platforms. The purpose of the review is not only to summarize prior work, but also to clarify what has already been optimized, what metrics are commonly used, and what gap remains open for STM32H7-class cryptographic offload.

A. Embedded Cryptographic Acceleration

A first body of work improves cryptographic performance by redesigning the accelerator itself. Zhang *et al.* address the real-time protection of streaming data by integrating a parallel AES-GCM engine into an embedded security SoC, showing that internal datapath parallelism can substantially increase throughput [5]. For public-key workloads, Agrawal *et al.* accelerate FPGA-based ECDSA verification through custom arithmetic pipelines and optimized elliptic-curve point operations [6]. The main contribution of this line of work is clear: accelerator microarchitecture strongly affects latency and throughput. Its main limitation, with respect to this proposal, is that the host-to-accelerator transfer path is usually assumed to be fixed or already efficient. As a result, these studies do not explain when an MCU should use direct I/O, DMA single-shot, or more aggressive buffered transfer strategies around the accelerator.

B. DMA and Interconnect Aware Embedded Design

A second theme studies the cost of moving data inside embedded systems. Vendor documentation and architecture guides show that DMA can reduce CPU intervention and improve burst efficiency when it is aligned with the underlying memory and interconnect structure [3], [4]. Bouajjani further highlights that DMA scheduling, locality, and transfer orchestration are themselves major performance factors in embedded multi-core systems [**?**]. Mera *et al.* add an important system-level perspective by showing that DMA is also a security-sensitive design choice because it introduces multi-master access behavior that affects isolation and protection mechanisms [**?**]. Together, these works motivate treating data movement as a first-class design variable rather than a transparent implementation detail. Their limitation is that they are largely general-purpose: they do not formulate cryptographic offload as a per-job transfer-mode selection problem, and they do not explicitly compare practical MCU-to-accelerator paths across different payload sizes and contention levels.

C. Adaptive and Reconfigurable Embedded Frameworks

A third line of work explores adaptive hardware/software execution. The ARTICo^3 framework shows how FPGA-based embedded systems can dynamically manage acceleration resources in cyber-physical settings, demonstrating the value of runtime-aware adaptation [**?**]. SmartDMA extends this perspective to programmable data movement, arguing that flexible memory-access engines are beneficial when workload behavior changes across edge scenarios [**?**]. Hardware/software co-design studies for modern cryptography, such as the work by Montanaro *et al.*, also show that end-to-end benefit depends not only on what is accelerated, but on how responsibilities are partitioned across software and hardware [**?**]. These works are highly relevant because they support the idea that static execution policies are often suboptimal. However, they usually operate at a coarser granularity than the present proposal: they focus on accelerator allocation, reconfiguration, or hardware/software partitioning in general, rather than on selecting the transfer mode of a concrete AES or ECDSA job running on an MCU-class host.

D. Positioning of This Work

Taken together, the literature suggests three conclusions. First, cryptographic accelerator design matters. Second, data movement and contention can dominate observed embedded performance even when the compute engine is efficient. Third, adaptive policies often outperform fixed strategies when workloads and resource conditions vary. The proposed work combines these insights at a narrower and more practical level. Rather than designing a new cipher core, it studies how an STM32H7-class host should choose among feasible transfer mechanisms when dispatching AES and ECDSA jobs to an accelerator. The distinguishing point is that host-to-accelerator data movement is treated as the optimization target, and the decision is evaluated using explicit system-level metrics: latency, CPU occupancy, and bus pressure.

TABLE I
COMPARISON OF REPRESENTATIVE RELATED STUDIES AND THE PROPOSED WORK.

Work	Year	Platform	Method / Focus	Metric / Result	Main Limitation w.r.t. This Proposal
Zhang *et al.* [5]	2021	Embedded SoC	Parallel AES-GCM accelerator datapath	Higher real-time stream throughput	Host-to-accelerator transfer path is not the optimization target
Agrawal *et al.* [6]	2022	FPGA accelerator	Pipelined ECDSA verification engine	Lower public-key verification latency	Limited MCU-side system context and transfer-path analysis
Saidi2012 *et al.* [7]	2012	Embedded multi-core	DMA scheduling and locality optimization	Better transfer efficiency and locality	Not cryptography-specific; no runtime transfer-mode selector
Mera *et al.* [8]	2022	Embedded systems	DMA-aware compartmentalization and isolation	Highlights DMA as a system/security concern	Security-centric; does not optimize AES/ECDSA execution paths
Rodrguez *et al.* [9]	2018	FPGA-based CPS/edge systems	Adaptive reconfigurable acceleration framework	Dynamic resource management benefits	Adaptation is coarser-grained than per-job MCU transfer selection
Zulberti *et al.* [10]	2024	Specialized edge architectures	Programmable memory-access controller	Adaptable data movement across workload classes	Not focused on MCU-hosted cryptographic jobs
Montanaro *et al.* [11]	2022	HW/SW crypto co-design	Accelerator/software partitioning for cryptography	Shows boundary management matters	PQC-focused; no explicit transfer-mode optimization
This work	2026	STM32H7 + DMA + accelerator	Per-job transfer-mode selection for AES/ECDSA	Latency, CPU occupancy, bus pressure	Experimental validation remains future work

III. PROBLEM DEFINITION

A. System / Application Description

The target application domain is a secure embedded node that invokes cryptographic operations for bulk protection and identity-related tasks. The reference platform is an STM32H745 class MCU with on-chip SRAM, a DMA engine, and a memory-mapped cryptographic accelerator interface. The accelerator may be external (for example FPGA-backed) or modeled as an internal peripheral-like block. Two workload families are considered:

- **AES jobs:** streaming-oriented bulk transfers where payload size ranges from short packets to large buffers.
- **ECDSA jobs:** smaller control-intensive transactions involving keys, digests, signatures, and command/status exchanges.

Each job arrives with a known algorithm label, payload size, and estimated contention class. The system must decide which transfer mode to use when dispatching the job to the accelerator.

B. Objectives

The primary objective is to choose a transfer mode that minimizes end-to-end cost for a cryptographic job. Cost is defined mainly by latency, but CPU occupancy and bus pressure are also important because they affect schedulability, responsiveness, and energy. The project therefore has four concrete objectives:

1) characterize the performance region in which each transfer strategy is preferable,
2) formulate transfer-mode selection as a constrained optimization problem,
3) design a lightweight selector that can run on an MCU-class system, and
4) compare adaptive selection against static baselines.

C. Constraints

The optimization is constrained by embedded-system realities:

- limited SRAM capacity and finite buffer sizes,
- DMA setup overhead and channel availability,
- bounded bus bandwidth and contention from concurrent traffic,
- strict response-time expectations for security-critical operations,
- practical implementation simplicity for an MCU runtime.

In particular, double buffering cannot be selected unless enough SRAM is available for two in-flight buffers plus metadata, and DMA-based modes are infeasible when the payload is too small to amortize setup cost.

D. Mathematical Formulation

Let J be the set of cryptographic jobs and $M = \{\text{PIO}, \text{DMA-SS}, \text{DMA-B}, \text{DBUF}\}$ the candidate transfer modes, corresponding to programmed I/O, DMA single-shot, DMA burst with batching, and double buffering. For job $j \in J$, define:

- $a_j \in \{\text{AES}, \text{ECDSA}\}$: algorithm type,
- s_j: payload size in bytes,
- $c_j \in \{L, M, H\}$: contention level,
- r_j: required response class or deadline category,
- $f_{j,m} \in \{0, 1\}$: feasibility of mode m for job j.

Let $x_{j,m} \in \{0, 1\}$ indicate whether mode m is selected for job j. The basic decision problem is:

$$\min \sum_{j \in J} \sum_{m \in M} x_{j,m} \cdot C(j, m) \tag{1}$$

subject to

$$\sum_{m \in M} x_{j,m} = 1 \qquad \forall j \in J \tag{2}$$

$$x_{j,m} \leq f_{j,m} \qquad \forall j \in J, \forall m \in M \tag{3}$$

The per-job cost is modeled as

$$C(j, m) = w_L \hat{L}(j, m) + w_U \hat{U}(j, m) + w_B \hat{B}(j, m) \tag{4}$$

where $\hat{L}(j, m)$ is normalized latency, $\hat{U}(j, m)$ is normalized CPU occupancy, and $\hat{B}(j, m)$ is normalized bus-pressure contribution. The weights w_L, w_U, and w_B can be tuned to represent system priorities, with w_L expected to dominate in real-time secure communication settings.

A practical latency model for profiling and selection is

$$L(j, m) = T_m^{\text{setup}} + T^{\text{move}}(a_j, s_j, c_j, m) + T^{\text{accel}}(a_j, s_j) + T_m^{\text{sync}} \tag{5}$$

where setup and synchronization costs depend on the chosen mode, while transfer and compute terms depend on workload type and contention. The optimization output is the mode $m^*(j)$ minimizing (4) for each job. In implementation terms, the exact optimizer is used offline to derive thresholds or to label training data; online execution uses a lightweight decision policy.

TABLE II
MAIN NOTATION USED IN THE OPTIMIZATION MODEL.

Symbol	Meaning
J	Set of cryptographic jobs
M	Candidate transfer modes
a_j	Algorithm of job j (AES or ECDSA)
s_j	Payload size of job j
c_j	Contention class of job j
r_j	Response-time or deadline class
$x_{j,m}$	Binary decision variable selecting mode m for job j
$f_{j,m}$	Feasibility indicator for mode m on job j
$C(j, m)$	Weighted cost of running job j with mode m
$L(j, m)$	End-to-end latency estimate
$\hat{U}(j, m)$	Normalized CPU occupancy term
$\hat{B}(j, m)$	Normalized bus-pressure term
w_L, w_U, w_B	Cost weights

IV. System Architecture and Design

A. Hardware/Software Architecture

The proposed system contains six main elements: application tasks, a runtime selector, the MCU execution core, SRAM buffers, a DMA engine, and the cryptographic accelerator. Background memory traffic is introduced to emulate realistic contention. The selector receives workload descriptors and chooses a transfer mode before the dispatcher interacts with the DMA engine or performs direct register writes.

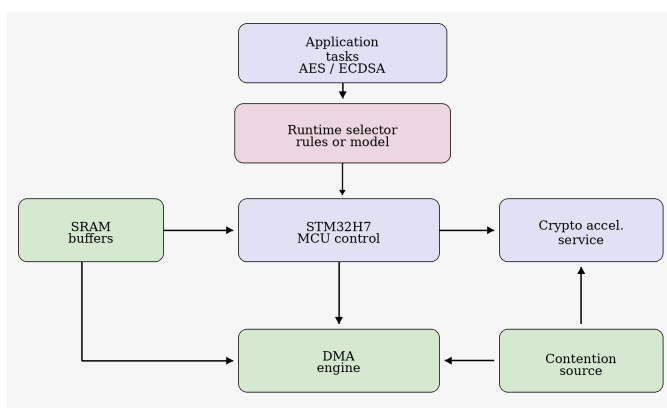

Fig. 1. Revised system architecture. The control path (application-selector-MCU-accelerator) is visually separated from the DMA-assisted bulk-transfer path to make the system structure clearer.

B. Dataflow and System Model

The system follows a request-dispatch-complete pattern. A job descriptor is produced by the application layer, the selector estimates the best mode, the MCU prepares descriptors

or control words, and the chosen path moves data to the accelerator. Completion can be interrupt-driven or polling-based depending on the mode and experiment. The dataflow is naturally represented as an event-driven control loop combined with block-level data transfers, which makes it suitable for a hybrid view: event-driven at the software control level and dataflow-oriented at the buffer movement level.

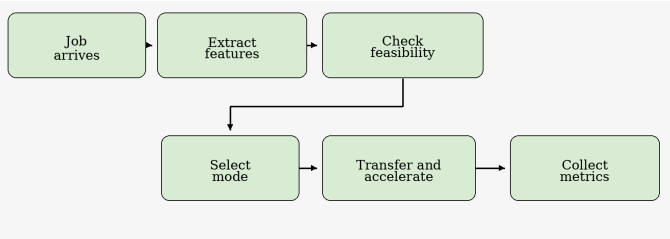

Fig. 2. Runtime dataflow of the proposed optimization pipeline. Collected metrics are later used offline to refine thresholds or lightweight models.

C. Timing and Execution Model

The runtime is modeled as an event-driven embedded controller. Cryptographic requests arrive asynchronously from higher-level software, are translated into compact descriptors, and are dispatched either through direct programmed I/O or through a DMA-assisted path. For AES-heavy traffic, the expected pattern is bursty and throughput-oriented; for ECDSA-heavy traffic, requests are smaller and more latency-sensitive. The selector therefore operates at job-arrival boundaries, where it can inspect payload size, request class, and current contention before committing to a mode. At evaluation time, the proposal will track both average latency and deadline-oriented metrics such as high-percentile latency or deadline-miss ratio for request classes with strict response requirements.

D. Model of Computation

A hybrid Model of Computation is the best fit for this project. The software control layer is naturally *event-driven*: job arrivals, DMA completion interrupts, timeout events, and accelerator completion events all trigger state transitions in the runtime. In contrast, the data-movement portion is closer to a *dataflow* model, because buffers, descriptors, and completion tokens move through a staged pipeline. This hybrid view is useful because it separates control decisions from bulk transfer mechanics. It also matches the optimization objective: the selector reasons about control-level events, but the cost model depends heavily on how data blocks traverse buffers, buses, and the accelerator interface.

E. Assumptions and Design Limitations

The proposal assumes that accelerator compute time is either measurable or can be modeled separately from transfer overhead. It also assumes that contention can be summarized into a small number of classes without fully modeling every bus event. This simplification is acceptable for a proposal-stage optimizer, but it limits accuracy when the memory system exhibits highly non-stationary behavior. Another limitation is that the selector optimizes per-job execution rather than full queue-level scheduling. A further assumption is that the runtime can observe enough state to classify contention approximately; if this observation is noisy, rule quality may degrade and more conservative policies may be required.

F. Resource Model

The relevant resources are SRAM, DMA availability, MCU cycles, and bus bandwidth. SRAM constrains buffering depth and whether double buffering is possible. DMA availability constrains when transfer overlap can be exploited. MCU utilization captures the cost of PIO-heavy approaches, while bus bandwidth determines whether large bursts improve efficiency or amplify interference. These resource limits are explicitly included in the feasibility function $f_{j,m}$ and in the measured cost model. In addition, the resource model tracks metadata overheads such as descriptor space, synchronization state, and temporary control buffers, because these hidden costs become significant on MCU-class devices even when the main payload is small.

V. Methodology

A. Optimization Pipeline

The project methodology contains five stages:

1) **Workload generation:** define AES and ECDSA job classes over different payload sizes and request patterns.
2) **Profiling:** measure or estimate latency, CPU occupancy, and contention sensitivity for each transfer mode.
3) **Offline optimization:** label each profiled case with the best mode according to the weighted objective in (4).
4) **Policy construction:** derive either threshold-based rules or a small classifier from the labeled dataset.
5) **Online evaluation:** execute the selector against unseen mixed workloads and compare it with static baselines.

This structure keeps the online logic simple while still grounding the decision policy in data.

B. Algorithm Description

The core online algorithm is intentionally lightweight so that it can run on an MCU without significant overhead. A rule-based version will be implemented first because it is easy to interpret and debug; an offline-trained decision tree is an optional extension if it provides better boundary handling.

Selector pseudocode

Input: job descriptor (a, s, c), resource state R
Feasible modes $F \leftarrow \text{check_feasibility}(a, s, c, R)$
for each $m \in F$ **do**
  estimate $\hat{L}(a, s, c, m)$
  estimate $\hat{U}(a, s, c, m)$
  estimate $\hat{B}(a, s, c, m)$
  compute score $S_m \leftarrow w_L \hat{L} + w_U \hat{U} + w_B \hat{B}$
end for
return $\arg\min_{m \in F} S_m$

Fig. 3. Online selection logic used at dispatch time.

The main implementation contribution of the project is the profiling and selector logic. Existing cryptographic kernels or a simplified accelerator stub may be reused, because the novelty is at the system-integration and decision-policy layer rather than in a new AES or ECDSA primitive.

VI. Experimental Setup

A. Platform and Tools

The reference target is an STM32H745 class platform, selected because it provides a realistic Cortex-M7 based embedded context with DMA support and constrained on-chip memory [12]. For proposal-stage evaluation, the plan is to use a timing-instrumented prototype consisting of C/C++ or Python-based workload drivers, a simplified memory-mapped accelerator model, and profiling scripts for offline analysis. If hardware access becomes available, the same logic can later be ported to an STM32H745 development board or a comparable platform.

B. Datasets / Workloads

The workloads are synthetic but system-relevant. AES cases will sweep payload sizes from short messages to larger buffers, while ECDSA cases will emphasize short descriptor-rich operations. Mixed traces will combine both job families under low, medium, and high contention. Background traffic will be created by concurrent memory copies or auxiliary DMA transfers to emulate interrupt-driven peripherals, streaming input, or competing tasks.

C. Evaluation Metrics and Baselines

The following baselines will be compared:

- always use PIO/MMIO,
- always use DMA single-shot,
- always use DMA burst/batching,
- oracle best mode from offline profiling,
- proposed adaptive selector.

The primary evaluation metrics are:

- end-to-end latency per job,
- throughput under mixed workloads,
- CPU occupancy or cycles spent in transfer-related control,
- estimated bus pressure or transferred bytes per useful byte,
- selector regret relative to the offline oracle,
- deadline-miss ratio for latency-sensitive classes,
- estimated energy proxy based on CPU active time and transfer count.

The evaluation plan will test three concrete hypotheses: (1) no single transfer mode dominates across all payload sizes and contention levels, (2) adaptive selection achieves lower cost than any static baseline under mixed workloads, and (3) most of the adaptive gain can be captured by a lightweight, explainable online policy rather than a heavy runtime optimizer. A successful outcome is therefore a selector that approaches oracle behavior while remaining lightweight, predictable, and robust across contention regimes.

VII. System-Level Considerations

This proposal explicitly considers system-level issues even where full implementation may be deferred.

Real-time constraints and latency: security operations frequently sit on critical paths such as authenticated boot, secure channel establishment, or packet protection. Therefore, the project prioritizes latency-aware mode selection and will report worst-case and percentile behavior, not only average times. Deadline-aware request classes are also included in the problem definition so that a transfer mode is not judged only by throughput.

Reliability and fault tolerance: DMA and buffering increase performance but also introduce error modes such as stale descriptors, incomplete transfers, synchronization bugs, and incorrect reuse of shared buffers. The design therefore includes completion checks, timeout handling, and conservative fallback to PIO when the resource state is uncertain. Reliability is especially important because a mismanaged transfer can silently corrupt cryptographic inputs or outputs.

Energy efficiency: although direct power measurement may not be available in the proposal phase, CPU occupancy and transfer count serve as useful proxies. Reducing software-driven copying is expected to reduce active CPU time, especially for large AES buffers.

Security considerations: because the project optimizes the movement path of sensitive material, security cannot be treated as external. Buffer lifetime, DMA permissions, and memory visibility matter, especially for keys, digests, and signatures. The proposal therefore includes buffer clearing, minimal exposure of secret data, and awareness that DMA changes the trust and isolation model [**?**]. The selector must never prefer a mode that violates a security precondition merely because it is faster.

Scalability: the immediate study focuses on AES and ECDSA, but the methodology is intended to generalize to other accelerator-backed operations as long as their feature vectors and cost profiles can be measured.

Implemented versus future work: the first implementation is expected to deliver profiling, offline labeling, a lightweight online selector, and measurement scripts. Queue-level optimization, richer contention predictors, direct energy instrumentation, and stronger security isolation mechanisms are positioned as future extensions.

Acknowledgments

The author would like to thank the course instructor for the valuable feedback that helped refine the scope, structure, and system-level framing of this proposal.

References

[1] NIST, "Advanced Encryption Standard (AES)," FIPS PUB 197, Nov. 2001, updated May 2023.
[2] NIST, "Digital Signature Standard (DSS)," FIPS PUB 186-5, Feb. 2023.
[3] Silicon Laboratories, "AN0013: Direct Memory Access," Application Note, 2013.
[4] Arm, "Introduction to AMBA AXI4," Issue 01, 2020.

[5] Z. Zhang *et al.*, "High-Efficiency Parallel Cryptographic Accelerator for Real-Time Data Security in Embedded Systems," *Micromachines*, vol. 12, no. 5, p. 560, 2021.

[6] R. Agrawal *et al.*, "Efficient FPGA-Based ECDSA Verification Engine for Permissioned Blockchains," in *Proc. ACM/SIGDA Int. Symp. on Field-Programmable Gate Arrays*, 2022.

[7] S. Saidi, P. Tendulkar, T. Lepley, and O. Maler, "Optimizing explicit data transfers for data parallel applications on the cell architecture," *ACM Transactions on Architecture and Code Optimization*, vol. 8, no. 4, pp. 1–20, Jan. 2012, doi: 10.1145/2086696.2086716.

[8] A. Mera, Y.-H. Chen, R. Sun, E. Kirda, and L. Lu, "D-Box: DMA-enabled compartmentalization for embedded applications," *arXiv preprint arXiv:2201.05199*, 2022, doi: 10.48550/ARXIV.2201.05199.

[9] A. Rodriguez *et al.*, "FPGA-Based High-Performance Embedded Systems for Adaptive Edge Computing in Cyber-Physical Systems: The ARTICo^3 Framework," *Sensors*, vol. 18, no. 6, p. 1877, 2018.

[10] G. Corradi, M. Cattaneo, and S. Di Carlo, "SmartDMA: Adaptable Memory Access Controller for Specialized Architectures in Edge Computing," preprint, 2024.

[11] G. Montanaro, A. Galimberti, E. Colizzi, and D. Zoni, "Hardware-Software Co-Design of BIKE with HLS-Generated Accelerators," arXiv:2209.03830, 2022.

[12] STMicroelectronics, "STM32H745xI/G Dual-Core Arm Cortex-M7 and Cortex-M4 MCUs Datasheet," Rev. 3, Jan. 2026.